

Ayudantía 8

Programación Funcional

Javier Larré

Pontificia Universidad Católica de Chile
javier.larre@uc.cl

01-05 de junio de 2026

Ayudantía de hoy

- 1 Intro a Funcional
- 2 Scheme
- 3 Ejercicios
- 4 Referencing

Lenguajes Funcionales

- Son lenguajes modelados a partir de las funciones matemáticas.

Lenguajes Funcionales

- Son lenguajes modelados a partir de las funciones matemáticas.
- En su forma más pura no utilizan variables, asignaciones ni cambios de estado.

Lenguajes Funcionales

- Son lenguajes modelados a partir de las funciones matemáticas.
- En su forma más pura no utilizan variables, asignaciones ni cambios de estado.
- Lo que implica que no utilizan *loops* ni *fors*, sino que recursión, listas y funciones para la iteración y el control de ejecución.

Lenguajes Funcionales

- Son lenguajes modelados a partir de las funciones matemáticas.
- En su forma más pura no utilizan variables, asignaciones ni cambios de estado.
- Lo que implica que no utilizan *loops* ni *fors*, sino que recursión, listas y funciones para la iteración y el control de ejecución.
- Las funciones más complejas se construyen con otras funciones, recursión y otras estructuras de datos, algunas con formas recursivas como las listas.

Lenguajes Funcionales

- Son lenguajes modelados a partir de las funciones matemáticas.
 - En su forma más pura no utilizan variables, asignaciones ni cambios de estado.
 - Lo que implica que no utilizan *loops* ni *fors*, sino que recursión, listas y funciones para la iteración y el control de ejecución.
 - Las funciones más complejas se construyen con otras funciones, recursión y otras estructuras de datos, algunas con formas recursivas como las listas.
- Si usamos funciones de primer orden en C++, ¿Entonces esto significa que C++ es funcional?

Lenguajes Funcionales

- Son lenguajes modelados a partir de las funciones matemáticas.
 - En su forma más pura no utilizan variables, asignaciones ni cambios de estado.
 - Lo que implica que no utilizan *loops* ni *fors*, sino que recursión, listas y funciones para la iteración y el control de ejecución.
 - Las funciones más complejas se construyen con otras funciones, recursión y otras estructuras de datos, algunas con formas recursivas como las listas.
- Si usamos funciones de primer orden en C++, ¿Entonces esto significa que C++ es funcional?
 - No, lenguajes que no son funcionales pueden escoger implementar algunas características de otros lenguajes o paradigmas.

- Son lenguajes modelados a partir de las funciones matemáticas.
 - En su forma más pura no utilizan variables, asignaciones ni cambios de estado.
 - Lo que implica que no utilizan *loops* ni *fors*, sino que recursión, listas y funciones para la iteración y el control de ejecución.
 - Las funciones más complejas se construyen con otras funciones, recursión y otras estructuras de datos, algunas con formas recursivas como las listas.
- Si usamos funciones de primer orden en C++, ¿Entonces esto significa que C++ es funcional?
 - No, lenguajes que no son funcionales pueden escoger implementar algunas características de otros lenguajes o paradigmas.
 - Otros ejemplos son las funciones lambda de Python o las queries de SQL (porque no permiten variables mutables en una query).

Otros ejemplos de lenguajes funcionales incluyen:

- Common Lisp
- Scheme (visto en clases)
- Clojure
- Wolfram (el lenguaje)
- Erlang
- Exilir
- OCaml
- Haskell
- F#

Scheme

Scheme is a classic programming language in the [Lisp family](#). It emphasizes [functional programming](#) and domain-specific languages but adapts to other styles. Known for its clean and minimalist design, Scheme is one of the longest-lived and best-studied [dynamic languages](#), and has many fast and portable implementations.

From the Scheme standard reports:

Programming languages should be designed not by piling feature on top of feature, but by removing the weaknesses and restrictions that make additional features appear necessary. Scheme demonstrates that a very small number of rules for forming expressions, with no restrictions on how they are composed, suffice to form a practical and efficient programming language that is flexible enough to support most of the major programming paradigms in use today.

The Scheme language, which is a dialect of Lisp, was developed at MIT in the mid-1970s (Sussman and Steele, 1975). It is characterized by its small size, its exclusive use of static scoping, and its treatment of functions as first-class entities. As first-class entities, Scheme functions can be the values of expressions, elements of lists, passed as parameters, and returned from functions. Early versions of Lisp did not provide all of these capabilities. As an essentially typeless small language with simple syntax and semantics, Scheme is well suited to educational applications, such as courses in

functional programming, and also to general introductions to programming. Most of the Scheme code in the following sections would require only minor modifications to be converted to valid Lisp code.

Vamos a programar!



Abran <https://try.scheme.org/> y ejecuten el siguiente código en la terminal REPL o en un archivo.

Para ejecutar un archivo deben escribir `(load "/ejemplo.scm")` en la terminal REPL.

Pueden encontrar ejemplos de scheme, y la solución de los ejercicios, en los archivos subidos a canvas.

Cosas importantes que deberían saber

```
;; Tipos Primitivos
```

```
3      ; Numeros  
1.02   ; Reales  
2/3    ; Fracciones  
#t     ; Verdad  
#f     ; Falso  
"hola" ; Strings  
'hola  ; Symbolos
```

```
;; Funciones Predefinidas
```

```
; + y * son funciones con N argumentos
```

```
(+ 1 2)  
(* 1 2 3)  
(sqrt 4)
```

```
; Componiendo Expresiones, and es otra funcion
```

```
(and (> 3 2) (equal? 1 1))
```

```
; Imprimir en Consola
```

```
(print "Hello, World!\n")
```

Pares y listas

```
(cons 1 2)
(car (cons 1 2)) ; extrae el primer elemento de un par
(cdr (cons 1 2)) ; extrae el segundo elemento de un par
;; Listas
;; en scheme la lista es una cadena de pares que termina en '()
'() ;; lista vacia
(cons 2 '()) ;; lista de un elemento
(cons 1 (cons 2 (cons 3 '()))) ;; lista de 3 elemento
(list 1 2 3) ;; otra forma de crear una lista
(append (list 1 2 3) (list 4 5 6)) ;; concatena dos listas
(cons 1 (list 2 3 4)) ;; agrega 1 al inicio de la lista 2 3 4
(car (list 1 2 3)) ;; saca el primer elemento de la lista
(cdr (list 1 2 3)) ;; devuelve una lista sin el primer elemento

(first (list 1 2 3)) ;; saca el primer elemento de la lista
(second (list 1 2 3)) ;; saca el segundo elemento de la lista
(third (list 1 2 3)) ;; saca el tercer elemento de la lista
(null? (list 1 2)) ;; verifica si la lista esta vacia
```

Identificadores

```
(define score 100)
```

```
(define (grade score)
  (if (> score 90)
      "A"
      (if (> score 80)
          "B"
          (if (> score 70)
              "C"
              "D")))))
```

```
(grade score)
```

```
(let ((precio 100)    ; Precio antes de impuestos
      (impuesto 0.19)) ; Tasa de impuesto del 19%
  (+ precio (* precio impuesto)))
```

Define la función `sum` que retorna la suma de los elementos de la lista entregada.

```
(define (sum lst)
  0
)

(test-equal (sum (list 1 1 1)) 3)
(test-equal (sum '()) 0)
(test-equal (sum (list 1)) 1)
(test-equal (sum (list 99 1)) 100)
```


Define la función `exist?` que retorna `true` si y solo si el elemento está en la lista.

```
(define (exist? elem lst)
  #t
)

(test-equal (exist? 1 (list 1 1 1)) #t)
(test-equal (exist? 0 (list 1 1 1)) #f)
(test-equal (exist? 0 '()) #f)
(test-equal (exist? 1 '()) #f)
(test-equal (exist? 1 (list 1)) #t)
(test-equal (exist? 0 (list 1)) #f)
```

Define la función `reverse` que retorna la lista entregada en orden invertido.

```
(define (reverse lst)
  '())

(test-equal (reverse '()) '())
(test-equal (reverse (list 1)) (list 1))
(test-equal (reverse (list 1 2)) (list 2 1))
(test-equal (reverse (list 1 2 3 4 5 6 7 8 9 10)) (list 10 9 8 7 6 5 4 3 2 1))
```

Define la función `insert` que retorna una lista equivalente a la entregada pero con el elemento entregado en la misma posición.

```
(define (insert elem lst)
  '())

(test-equal (insert 1 '()) (list 1))
(test-equal (insert 2 (list 1)) (list 1 2))
(test-equal (insert 3 (list 1 2)) (list 1 2 3))
(test-equal (insert 4 (list 1 2 3)) (list 1 2 3 4))
```

Define la función `delete` que retorna una lista equivalente a la entregada pero con el elemento entregado eliminado de la lista.

```
(define (delete elem lst)
  )

(test-equal (delete 1 '()) '())
(test-equal (delete 1 (list 1)) '())
(test-equal (delete 2 (list 1 2)) (list 1))
(test-equal (delete 3 (list 1 2 3)) (list 1 2))
(test-equal (delete 5 (list 1 2 3 4 5 6 7 8 9 10)) (list 1 2 3 4 6 7 8 9 10))
(test-equal (delete 10 (list 1 2 3 4 5 6 7 8 9 10)) (list 1 2 3 4 5 6 7 8 9))
```

Define la función `filter-evens` que retorna una lista con los pares de la lista entregada.

```
(define (filter-evens lst)
  '())

(test-equal (filter-evens (list 1 2 3 4)) (list 2 4))
```

Define la función `filter` que retorna una lista con los elementos de la lista entregada que cumpla con la condición entregada.

```
(define (filter condition lst)
  '())

(test-equal (filter even? (list 1 2 3 4)) (list 2 4))
(define (is-true? value) value)
(test-equal (filter is-true? (list #t #t #f)) (list #t #t))
```

Define la función `fold` que retorna el resultado de aplicar la función entregada sobre la lista acumuladamente. Es equivalente a `reduce` en Python.

```
(define (fold func initial-value lst)
  initial-value
)

(test-equal (fold + 0 (list 1 2 3)) 6)
(define (and-func a b) (and a b))
(test-equal (fold and-func #t (list #t #t #t)) #t)
(test-equal (fold and-func #t (list #t #t #f)) #f)
```

Define la función `collect` que retorna una lista donde cada elemento es el resultado de aplicar la función entregada un elemento de la lista. Es equivalente a `map` en Python.

```
(define (collect function lst)
  (if (null? lst)
      '()
      (cons (function (car lst)) (collect function (cdr lst)))))

(define (fun-add-two n) (+ n 2))
(test-equal (collect fun-add-two (list 1 2 3)) (list 3 4 5))

;; Using anonymous function (lambda) for addition
(test-equal (collect (lambda (n) (+ n 2)) (list 1 2 3)) (list 3 4 5))

;; Using anonymous function (lambda) for multiplication
(test-equal (collect (lambda (n) (* n 2)) (list 1 2 3)) (list 2 4 6))
```


Define la función `fun-add` que retorna una función que suma el número entregado a otro número.

```
(define (fun-add k)
  0
)
```

```
(define add2 (fun-add 2))
(define add4 (fun-add 4))
(test-equal (add2 1) 3) ; le suma dos a uno
(test-equal (add4 1) 5) ; le suma cuatro a uno
```

References



Robert Sebesta (2023)

Concepts of programming languages

The End

Questions? Comments?